```
// Kurulum: npm i ws   |   Çalıştır: node server.js   |   sohbet.html bu dosyayla
aynı klasörde olmalı
const http = require('http'), fs = require('fs'), path = require('path');
const { WebSocketServer } = require('ws');
const DB = path.join(__dirname, 'db.json');
let db = { users: {}, queue: {}, blocks: {} };
try { db = JSON.parse(fs.readFileSync(DB, 'utf8')); } catch (e) {}
const save = () => fs.writeFile(DB, JSON.stringify(db), () => {});

const srv = http.createServer((q, r) => {
  r.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
  fs.createReadStream(path.join(__dirname, 'sohbet.html')).pipe(r);
});
const wss = new WebSocketServer({ server: srv, maxPayload: 12e6 });
const online = new Map(); // tel -> ws
const push = (tel, o) => { const w = online.get(tel); if (w && w.readyState === 1) {
w.send(JSON.stringify(o)); return true; } return false; };
const info = t => ({ tel: t, reg: !!db.users[t], on: online.has(t), seen:
(db.users[t] || {}).seen || 0 });

wss.on('connection', ws => {
  let me = null;
  ws.on('message', raw => {
    let m; try { m = JSON.parse(raw); } catch (e) { return; }
    if (m.t === 'login') {
      me = String(m.tel || '').replace(/\D/g, '');
      if (me.length < 8) return;
      online.set(me, ws);
      db.users[me] = db.users[me] || {};
      ws.send(JSON.stringify({ t: 'ready', me }));
      (db.queue[me] || []).forEach(q => { ws.send(JSON.stringify(q)); push(q.from, {
t: 'ack', to: me, id: q.id, s: 2 }); });
      delete db.queue[me]; save();
      online.forEach((w, t) => { if (t !== me) push(t, Object.assign({ t: 'pres' },
info(me))); });
      return;
    }
    if (!me) return;
    if (m.t === 'lookup') { ws.send(JSON.stringify({ t: 'lookup', list: (m.tels ||
[]).map(info) })); return; }
    if (m.t === 'msg') {
      const to = String(m.to);
      if (!db.users[to]) { push(me, { t: 'err', e: 'Bu numara Sohbet'te kayıtlı
değil' }); return; }
      if ((db.blocks[to] || []).includes(me)) return; // engelli: gönderen tek tikte
kalır
```

```
      const out = { t: 'msg', from: me, id: m.id, text: m.text, img: m.img, file:
m.file, exp: m.exp, ts: Date.now() };
      if (push(to, out)) push(me, { t: 'ack', to, id: m.id, s: 2 });
      else { (db.queue[to] = db.queue[to] || []).push(out); save(); }
      return;
    }
    if (m.t === 'read') { push(m.to, { t: 'read', from: me, ids: m.ids }); return; }
    if (m.t === 'del') {
      if (db.queue[m.to]) { db.queue[m.to] = db.queue[m.to].filter(q => q.id !==
m.id); save(); }
      push(m.to, { t: 'del', from: me, id: m.id }); return;
    }
    if (m.t === 'block') {
      const b = db.blocks[me] = db.blocks[me] || [], i = b.indexOf(m.tel);
      if (m.on && i < 0) b.push(m.tel);
      if (!m.on && i > -1) b.splice(i, 1);
      save();
    }
  });
  ws.on('close', () => {
    if (me && online.get(me) === ws) {
      online.delete(me); db.users[me].seen = Date.now(); save();
      online.forEach((w, t) => push(t, Object.assign({ t: 'pres' }, info(me))));
    }
  });
});
srv.listen(process.env.PORT || 8080, () => console.log('Sunucu hazır:
http://localhost:8080'));
```